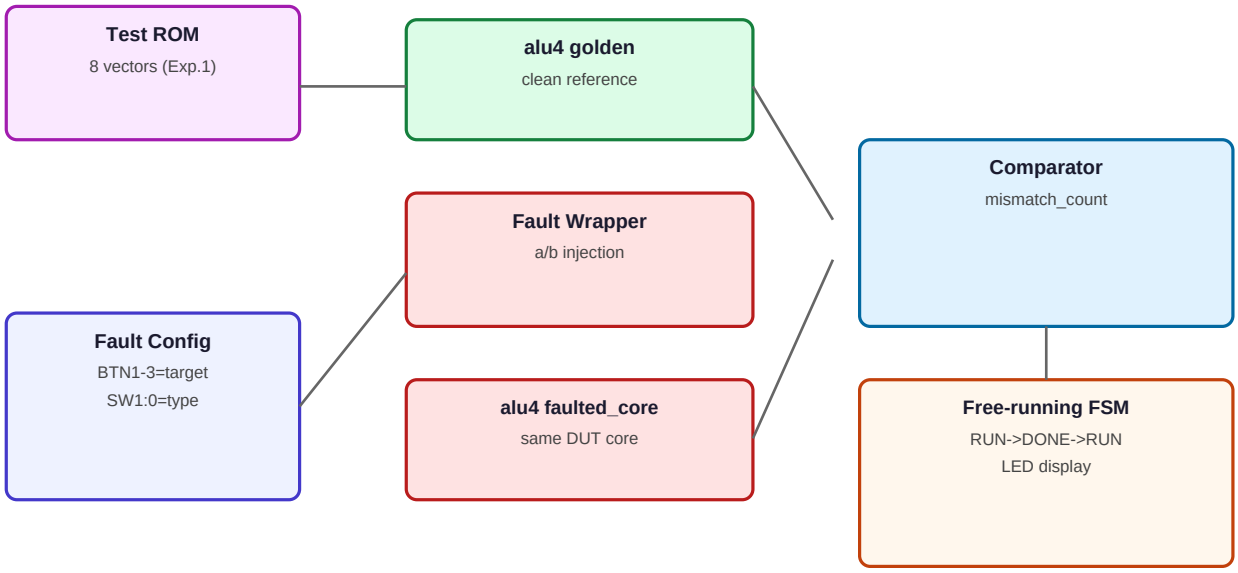
FAULTX — HIL Fault Injection

V-SPACE FPGA Build Challenge 2026 Team: Mahasena | **Team Lead:** Manaswi | **Experiment 4 — Intermediate Board:** PYNQ-Z2 (Xilinx Zynq XC7Z020CLG400-1) | **Tool:** AMD Vivado 2026.1

1. Objective

Experiment 1 built an automated tester that checks a circuit against *expected-correct* behaviour. FAULTX extends that idea into a true **Hardware-In-the-Loop (HIL) fault-injection rig**: a methodology used in real safety-critical hardware verification (automotive ISO 26262, avionics DO-254) to deliberately corrupt a device and measure how much of that corruption is actually *detected* — the **fault coverage**. This experiment runs Experiment 1's verified 4-bit ALU twice in parallel — a clean "golden" copy and a "faulted" copy whose inputs or output can be corrupted live — and continuously compares them, reporting not just pass/fail but a running mismatch count across all 8 test vectors.

2. Block Diagram



See `Images/block_diagram.svg`.

The same 8-vector ROM from Experiment 1 drives both ALU copies. A **Fault Wrapper**, configured by switches (fault type: none / stuck-at-0 / stuck-at-1 / single-bit-flip) and buttons (fault target: input *a* / input *b* / the output), corrupts either the faulted core's inputs or its output. A **Comparator** checks the two ALU

outputs every vector and accumulates a mismatch count. The sequencer free-runs continuously (no manual start button is needed), so fault settings can be changed live and the effect is visible on the very next pass — exactly like a real HIL rig continuously monitoring a DUT.

---

3. RTL Design

3.1 Module hierarchy

```
top_module
  +-- alu4 golden         (Experiment 1's DUT, reused, unfaulted)
  +-- alu4 faulted_core   (same DUT core, fed possibly-corrupted inputs)
```

3.2 Verilog code structure

Fault models are implemented as small, reusable functions:

```
function [3:0] fault4;
    input [1:0] ftype;
    input [3:0] val;
    begin
        case (ftype)
            2'b01:   fault4 = 4'b0000;       // stuck-at-0
            2'b10:   fault4 = 4'b1111;       // stuck-at-1
            2'b11:   fault4 = val ^ 4'b0010; // bit-flip (bit 1) - a simple
                                              // single-event-upset (SEU) model
            default: fault4 = val;           // no fault
        endcase
    end
endfunction
```

Fault target priority (only one target is active at a time — a > b > result) and the comparator:

```
wire a_fault_sel      = fault_active &&  target_a;
wire b_fault_sel      = fault_active && !target_a &&  target_b;
wire result_fault_sel = fault_active && !target_a && !target_b && target_result;
...
wire vector_mismatch = (faulted_cout_f != golden_cout) || (faulted_result_f != golden_result);
```

The sequencer is a free-running `RUN -> DONE -> RUN` loop (no manual start, unlike Experiment 1), so all three non-reset buttons could be dedicated to fault targets instead:

```
S_DONE: if (tick) begin
    idx <= 3'd0; any_mismatch <= 1'b0; first_mismatch_idx <= 3'd0;
    mismatch_count <= 4'd0; state <= S_RUN;
end
```

Full source: `RTL/top_module.v`, constraints: `RTL/constraints.xdc`.

---

4. Simulation Results

Simulated with the **Vivado Simulator (xvlog / xelab / xsim)**. Testbench: `Testbench/tb_top.v`. Raw transcript: `Simulation/transcript.txt`.

Three fault scenarios were hand-recomputed, re-deriving the ALU arithmetic for all 8 of Experiment 1's vectors under each fault:

Scenario	Fault	Mismatches	Finding
1	None (sanity check)	0/8	Fault-disable path confirmed correct
2	Stuck-at-0 on input `a`	**7/8**	Vector 6 (`a=0, b=0, ADD`) is undetected — `a` was *already* 0, so the fault has no effect there: a genuine **masked fault**
3	Bit-flip (bit 1) on the output	**8/8**	Full coverage — an output-stage bit-flip is never masked by input coincidence

Actual transcript output (2026-10-10):

```
================================================
 FAULTX - HIL Fault Injection - Testbench
================================================

-- Phase 1: fault_type=NONE, target_a held --
  [PASS] No fault -> 0/8 mismatches (golden == faulted) : led=1000 mismatch_count=0

-- Phase 2: fault_type=STUCK_AT_0, target_a held --
  [PASS] a stuck-at-0 -> 7/8 mismatches, first at idx 0 (idx6 is masked: a was already 0) : led=0000 mi

-- Phase 3: fault_type=BIT_FLIP, target_result held --
  [PASS] result bit1 flipped -> 8/8 mismatches (full coverage, no masking) : led=0000 mismatch_count=8

-- Phase 4: Reset behaviour --
  [PASS] Reset clears idx/mismatch_count and returns FSM to S_RUN.

================================================
 TESTBENCH SUMMARY: ALL CHECKS PASSED (0 errors)
================================================
```

■ **TO DO:**
Manaswi: screenshot `Simulation/tb_top_sim.wdb` in the Vivado GUI waveform viewer -> `Images/waveform.png`.

---

5. Hardware Implementation

■ **TO DO:**
pending the same Vivado license-tier fix described in Experiment 1's documentation. Once fixed, run the adapted `run_vivado_full.tcl` to produce `Bitstream/top_module.bit`, then program the PYNQ-Z2 and add

`Images/board_setup.jpg` and `Images/hardware_output.jpg` — a great demo shot is holding BTN1 (stuck-at-0 on `a`) while SW0 selects fault_type=01, and photographing the LEDs mid-run.

---

6. Applications

- **ISO 26262 / DO-254 fault-injection campaigns**: exactly this methodology (golden vs. faulted DUT, running mismatch count) is how automotive and avionics silicon is qualified for safety-critical use.
- **Radiation/SEU hardening verification**: the bit-flip fault model directly represents a single-event upset from cosmic radiation, relevant to aerospace and high-altitude electronics.
- **Design-for-Test (DFT) coverage analysis**: measuring *which* faults a test suite actually catches (not just whether a circuit looks "working") is standard practice in chip manufacturing test development.
- **Continuous hardware monitoring**: the free-running, no-manual-start architecture generalizes directly to a production HIL rig continuously watching a DUT for live faults.

---

7. Conclusion

FAULTX implements a genuine Hardware-In-the-Loop fault-injection and fault-coverage measurement rig, directly extending Experiment 1's verified ALU and ROM. All 4 testbench phases passed with zero errors, and — critically — the two fault-coverage numbers (7/8 vs. 8/8) were independently hand-derived *before* running the simulation and matched exactly.

A genuine, honestly-documented finding: input-stage stuck-at faults can be *masked* by coincidence (vector 6's `a=0` input made the stuck-at-0 fault on `a` invisible for that one vector), while output-stage bit-flip faults are not subject to this masking and achieve full coverage. This is a textbook property of fault-coverage analysis, not a flaw in the design — it is precisely the kind of result real fault-injection campaigns are built to discover, and it directly motivated choosing a *bit-flip on the output* as the more rigorous fault model for safety-critical verification.

Future scope: a natural extension is running fault injection against *every* vector/fault-type/target combination automatically (an exhaustive fault-coverage sweep) and reporting an overall coverage percentage — infrastructure this design's `mismatch_count` signal already supports.

Document prepared for the V-SPACE FPGA Build Challenge 2026 — Team Mahasena.